



UNIVALI

Universidade do Vale do Itajaí
Escola do Mar, Ciência e Tecnologia - EMCT
Ciência da Computação

RECONHECIMENTO DE IMAGENS: ROCK PAPER SCISSORS

Gustavo Henrique Stahl Müller
Paulo Henrique Rohling
Caio Gonzaga Bernils

Índice

1. Aplicação
2. Dataset
3. Pré-processamento
4. Classificador SVC
5. Treinamento
6. Resultados do treinamento
7. Demonstração

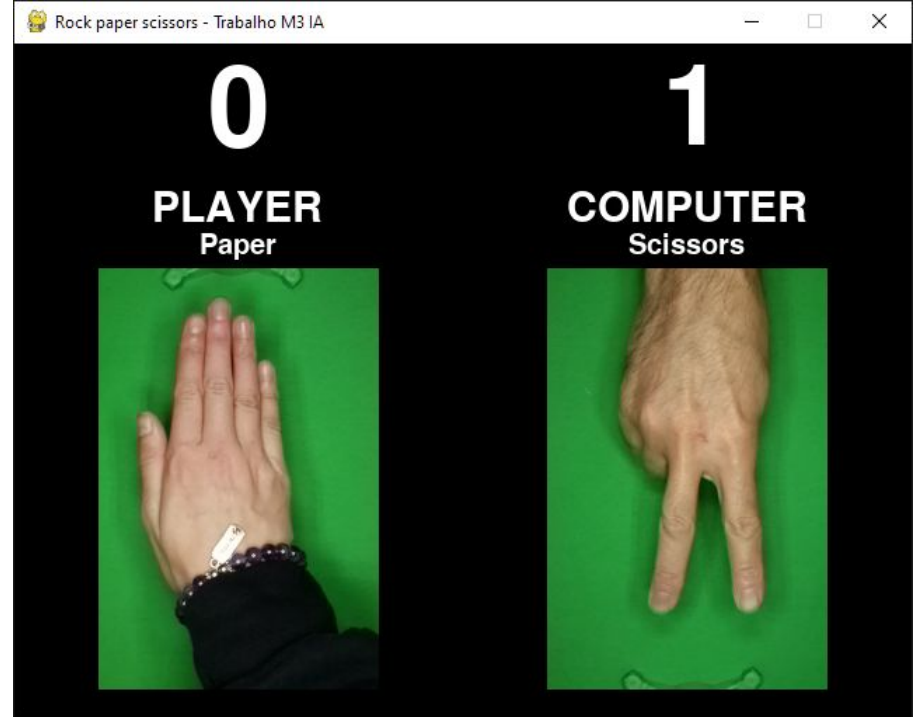
Aplicação



- A aplicação desenvolvida é um jogo de pedra, papel e tesoura que utiliza reconhecimento de imagem para identificar os gestos presentes no jogo.

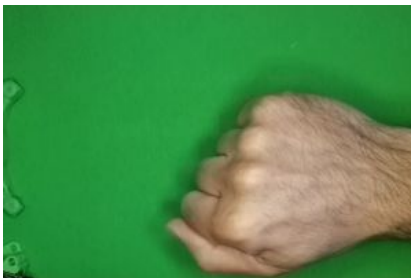
Aplicação

- Foram utilizados:
 - Python 3.11
 - Numpy
 - OpenCV
 - Pygame
 - Scikit-learn



Dataset

- **Rock-Paper-Scissors Images**, disponibilizado no Kaggle;
- No total possui:
 - 163.71 MB de tamanho;
 - 2188 imagens (726 pedras, 710 papéis e 752 tesouras);
- As imagens estão em formato **.png**, com 300x200 de resolução;
- Todas as imagens possuem um fundo verde com iluminação consistente.



Pré-processamento

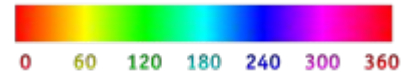
- Objetivo de melhorar o contraste entre a mão e o fundo da imagem;
- Aplicada em cada imagem do dataset;
- Dividido em 4 etapas:
 1. **RGB para BGR:** Transforma para BGR (formato usado pelo OpenCV);
 2. **Color distance:** Transforma a imagem de BGR para HSV e realiza a diferença entre o canal H e um valor pré-estabelecido (63);
 3. **Threshold Simples:** Remove o fundo da imagem;
 4. **Grayscale:** Transforma a imagem para escala de cinza.

Color Distance

- Objetivo de identificar o que é mão ou fundo;
- Resultado (imagem com um canal) será usado na etapa seguinte.

```

85 | def hueDistance(image, hueValue):
86 |     # Convert image to HSV colorspace
87 |     hsv = cv2.cvtColor(image, cv2.COLOR_RGB2HSV)
88 |     hChannel = hsv[:, :, 0].astype(int)
89 |
90 |     dist = np.minimum(np.abs(hChannel - hueValue),
91 |                       | 360 - np.abs(hChannel - hueValue))
92 |
93 |     return dist
  
```



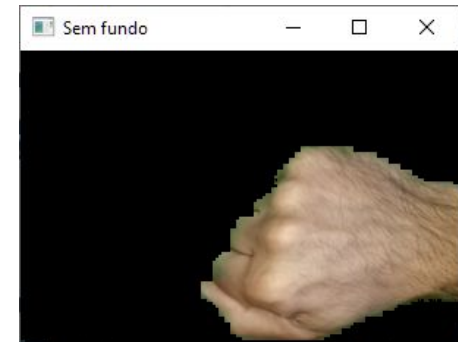
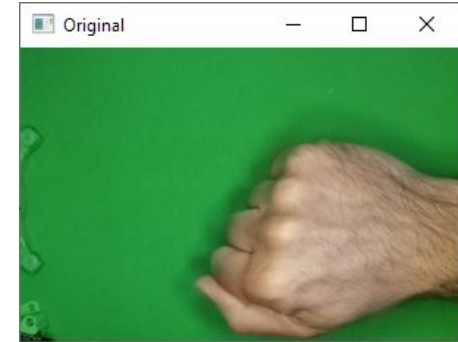
Threshold Simple

- Objetivo de retirar o fundo da imagem

```

96 def removeBackground(image, hueValue, threshold = 0):
97     # Get an image corresponding to the hue distance from the background hue value
98     dist = hueDistance(image, hueValue)
99
100    # Create a copy of the source image to use as masked image
101    masked = image.copy()
102
103    # Select background pixels using thresholding and set value to zero (black)
104    if threshold == 0:
105        masked[dist < filters.threshold_mean(dist)] = 0
106    else:
107        masked[dist < threshold] = 0
108
109    return masked
110

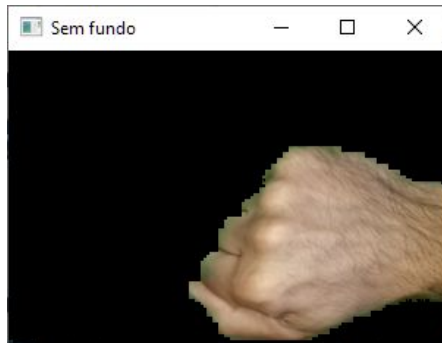
```



Grayscale

- Objetivo de reduzir o impacto de diferenças de cores

```
67 def getGray(image, hueValue = 63, threshold = 0):  
68     image = removeBackground(image, hueValue, threshold)  
69     image = cv2.cvtColor(image, cv2.COLOR_RGB2GRAY).astype(np.float32) / 255  
70  
71     return image.ravel()
```

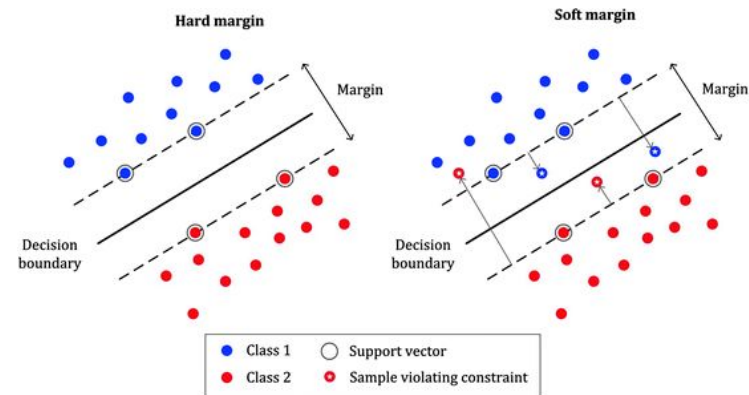


Classificador SVC

- SVC: Support Vector Classifier;
- Classificador supervisionado de aprendizado de máquina;
- Tenta separar o conjunto de dados de treino com um hiperplano;

Foi usado o classificador **SVC** com kernel **RBF**;

- RBF: Radial Basis Function;



Classificador SVC

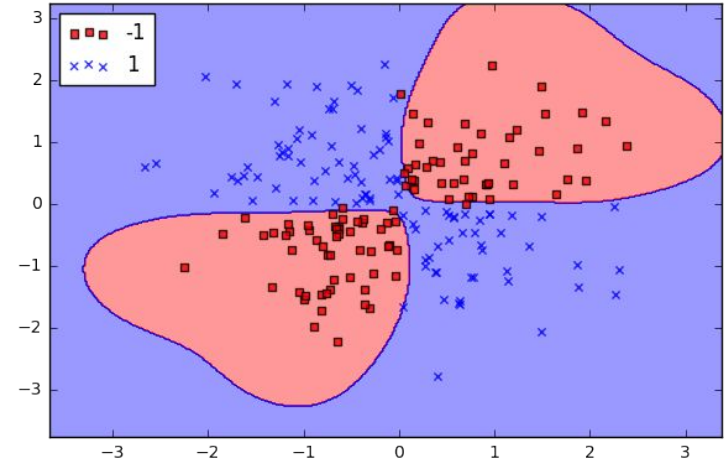
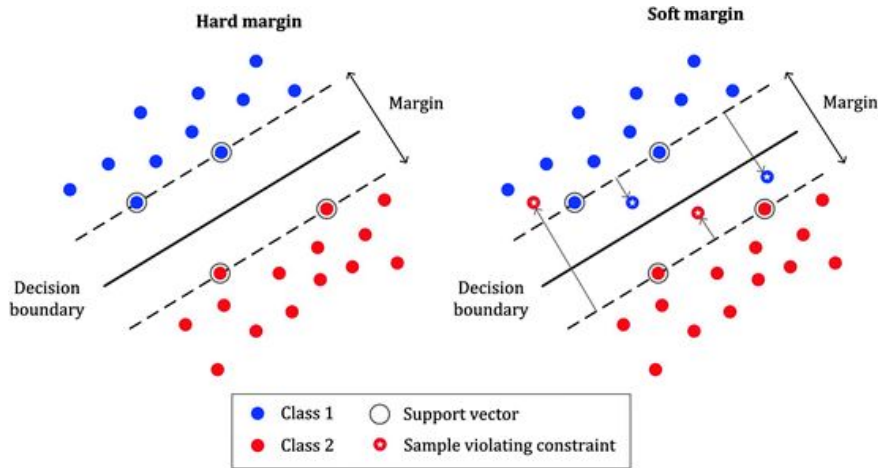
- Inicialmente foi testado o classificador **SVC** com kernel **linear**;
 - Não rendeu bons resultados;

```
Confusion matrix:
[[62  2  8]
 [ 3 59  9]
 [ 5  2 68]]
Classification report:
              precision    recall
rock          0.89         0.86
paper         0.94         0.83
scissors      0.80         0.91

accuracy
macro avg    0.87         0.87
weighted avg 0.87         0.87
```

Classificador SVC

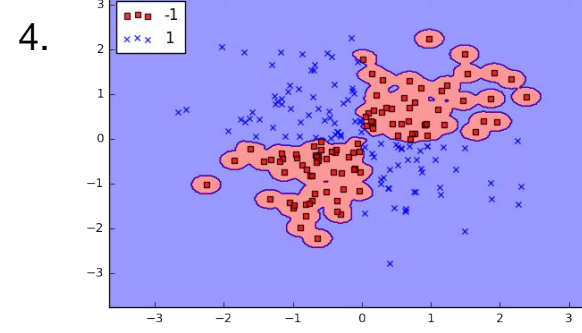
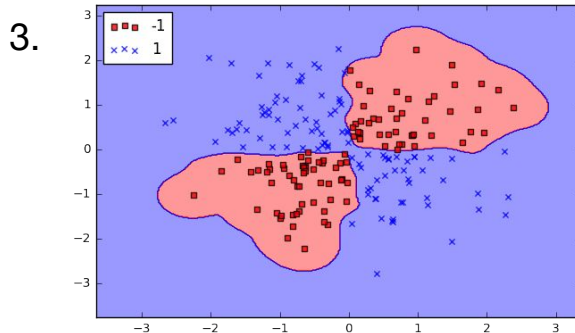
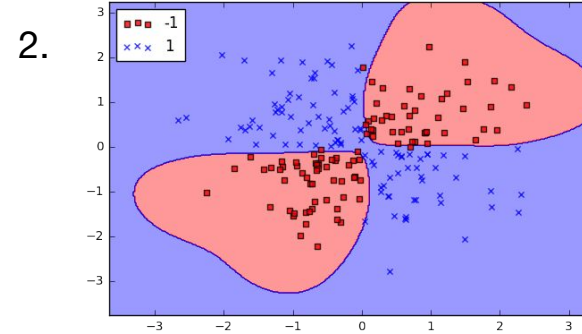
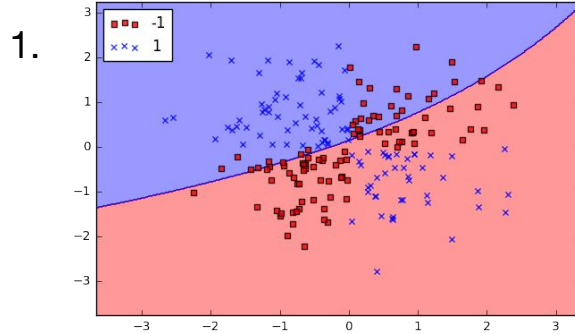
- Inicialmente foi testado o classificador **SVC** com kernel **linear**;
 - Não rendeu bons resultados;



Hiperparâmetros do Classificador

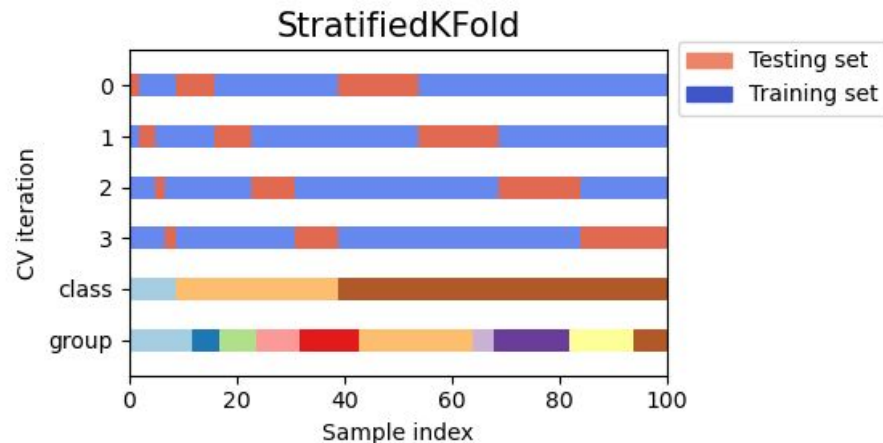
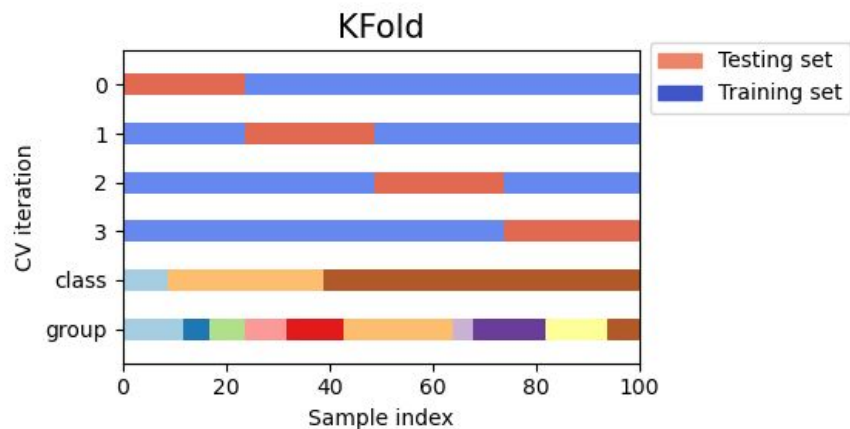
- Gamma: Quanta influência um datapoint tem na criação dos limites;
 - Valores testados: [.0001, .001, .01];
- C: O quão penalizado o classificador é por classificar algo de maneira errada;
 - Valores testados: [1, 10, 100];
- Foi feita uma busca exaustiva de todas as combinações possíveis desses valores para encontrar as melhores combinações (**GridSearchCV**);

Hiperparâmetros do Classificador



Treinamento

- Uso do validador **StratifiedKFold**;



```
57 # Generate test set
58 print('+{}s: Generating test set'.format(dt()))
59 stratifiedKFold = StratifiedKFold(n_splits = 10, shuffle = True, random_state = rs)
```

Treinamento

- Foi usado PCA para reduzir o tamanho dos dados antes do fit do classificador:
 - Inicialmente testamos compressão de imagens ao invés de PCA;
 - Não rendeu bons resultados;

```
62     # Define pipeline parameters
63     print('+{}s: Defining pipeline'.format(dt()))
64     steps = [('pca', PCA()), ('clf', SVC(kernel = 'rbf'))]
65     pipeline = Pipeline(steps)
```


Treinamento

- Para otimizar os hiperparâmetros e fazer o fit do classificador, foi usado o **GridSearchCV**:

```
# Define cross-validation parameters
print('+{}s: Defining cross-validation'.format(dt()))
crossValidator = StratifiedKfold(n_splits = nSplits, shuffle = True, random_state = randomState)

# Define grid-search parameters
print('+{}s: Defining grid search'.format(dt()))
gridParams = dict(pca__n_components = pca__n_components,
                  clf__gamma = clf__gamma,
                  clf__C = clf__C)

grid = GridSearchCV(pipeline, gridParams, scoring = scoring,
                    n_jobs = nJobs, refit = True, cv = crossValidator, verbose = 1)
```

Treinamento

- Por último, é feito o fit do classificador e ele é aplicado no conjunto de testes separado anteriormente:

```
83     # Fit the classifier
84     t0_train = time.time()
85     print('+{}s: Fitting classifier'.format(dt()))
86     grid.fit(features_train, labels_train)
87     dt_train = time.time() - t0_train
```

```
105 | prediction = grid.predict(features_test)
106 | score = f1_score(labels_test, prediction, average = 'micro')
```

Resultados do treinamento

```
Classifier f1-score on test set: 0.9908256880733946
Confusion matrix:
[[72  0  0]
 [ 1 70  0]
 [ 0  1 74]]
Classification report:
```

	precision	recall	f1-score	support
rock	0.99	1.00	0.99	72
paper	0.99	0.99	0.99	71
scissors	1.00	0.99	0.99	75
accuracy			0.99	218
macro avg	0.99	0.99	0.99	218
weighted avg	0.99	0.99	0.99	218

Referências



HU, M-K. **Visual Pattern Recognition by Moment Invariants**. Tese (Mestrado em Engenharia Elétrica). Disponível em: <<http://www.sci.utah.edu/~gerig/CS7960-S2010/handouts/Hu.pdf>>. Acesso em 30 out. 2022.

MALLICK, S. **Image Recognition and Object Detection**. 2016. Disponível em: <<https://learnopencv.com/image-recognition-and-object-detection-part1/>>. Acesso em 30 out. 2022.

PEER, P., KOVAC, J., SOLINA, F. **Human Skin Colour Clustering for Face Detection**. Disponível em: <<https://tinyurl.com/46zh7j7/>>. Acesso em 30 out. 2022.

RIBEIRO, H. L. **Reconhecimento de gestos usando segmentação de imagens dinâmicas de mãos baseada no modelo de mistura de gaussianas e cor de pele**. Tese (Mestrado em Engenharia Elétrica) – Escola de Engenharia de São Carlos, Universidade de São Paulo. São Carlos, 2006.

TERREAULT, J. B. **RPS-CV**. Disponível em: <<https://github.com/DrGFreeman/rps-cv>>. Acesso em 30 out. 2022.

MATHWORKS. **What Is Image Recognition? 3 things you need to know**. Disponível em: <<https://www.mathworks.com/discovery/image-recognition-matlab.html>>. Acesso em 1 nov. 2022.



UNIVALI

Universidade do Vale do Itajaí
Escola do Mar, Ciência e Tecnologia - EMCT
Ciência da Computação

RECONHECIMENTO DE IMAGENS: ROCK PAPER SCISSORS

Gustavo Henrique Stahl Müller
Paulo Henrique Rohling
Caio Gonzaga Bernils
